

Objectives for this session:

To briefly explore and explain
Quantum Machine Learning with
PennyLane hands-on demos
all in less than 60 mins



What is QML?

Parameterised Quantum Circuits (PQC)

Variational Quantum Algorithms (VQA)

Quantum data encoding

Quantum state measurement

Why using PennyLane?

Training a PennyLane model

Sample model training

- Why did it succeed and then failed?
- Data encoding nightmares
- How was the model improved?

QML readings

Summary

Introduction to QML+ (with hands-on in PennyLane)

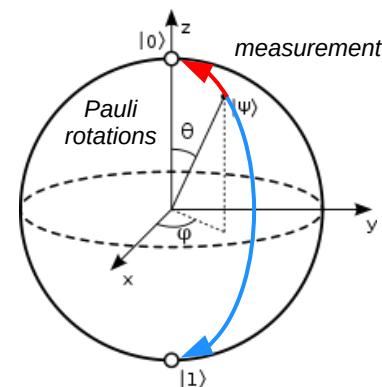
Jacob L. Cybulski

Enquanted, Australia /
School of IT, Deakin University

and

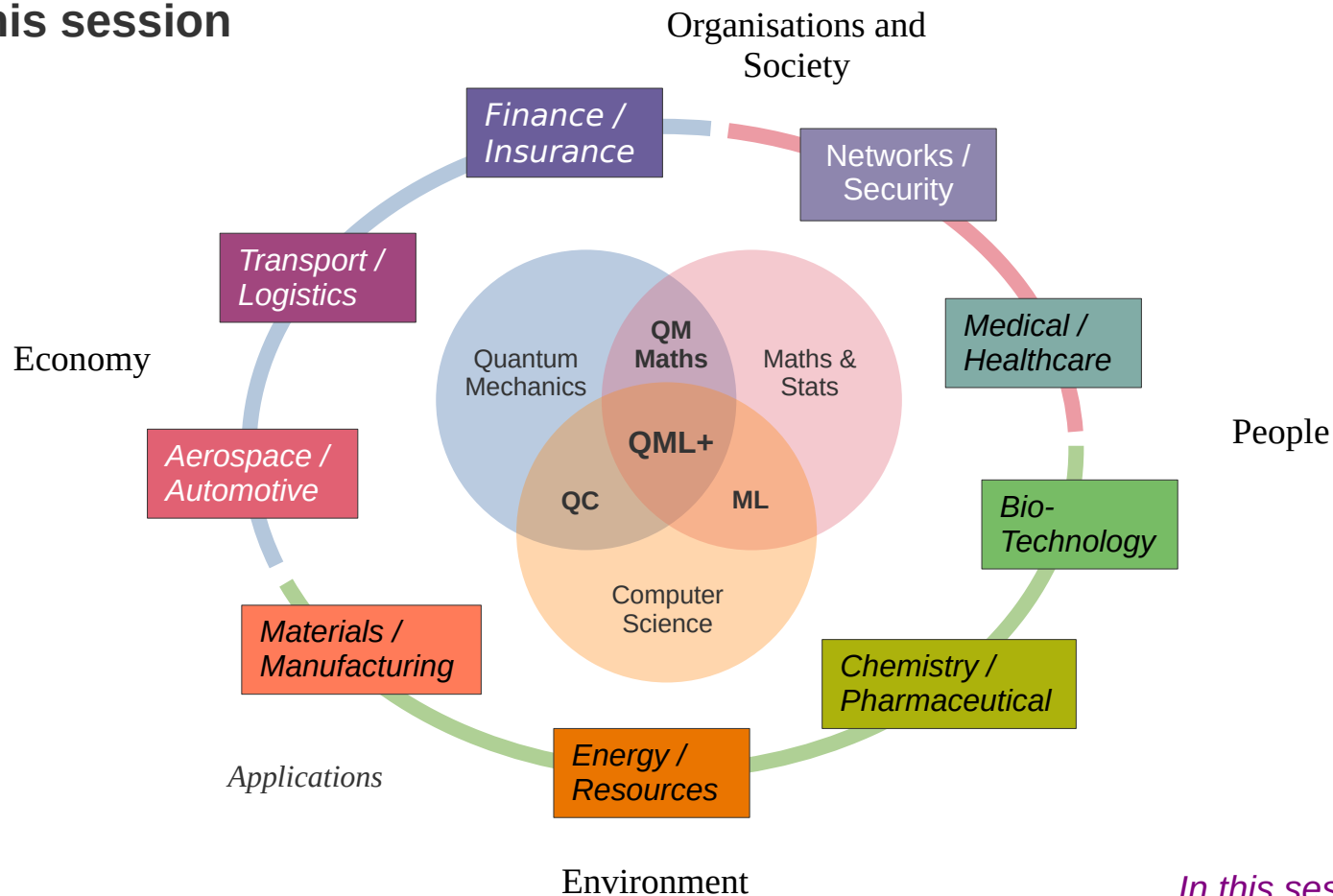
Fatima Ansarizadeh

School of IT, Deakin University



Quantum ML+

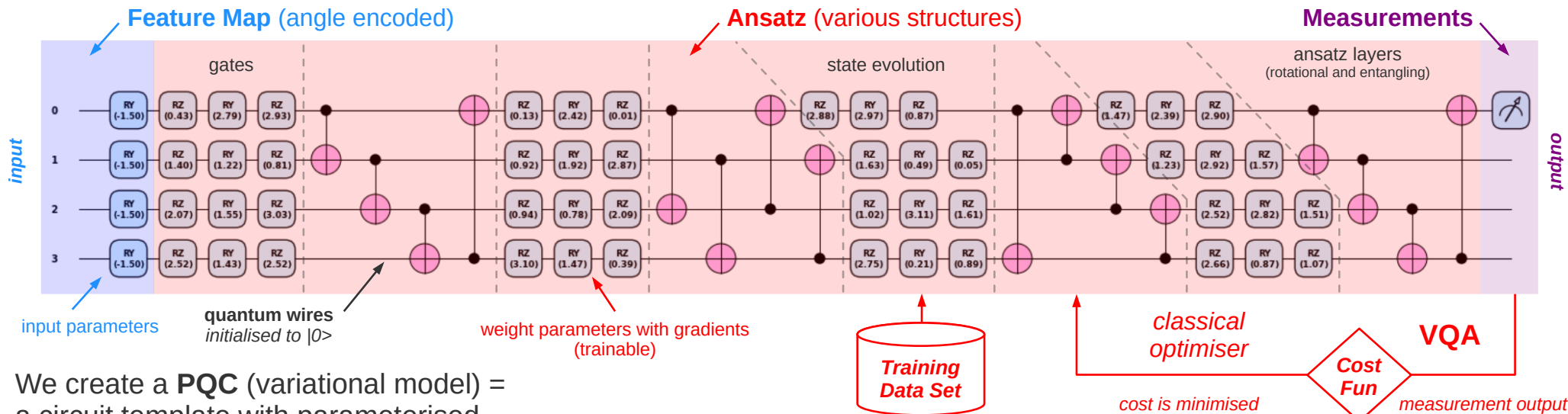
aims of this session



*In this session we will explain
the practice of QML using PennyLane*

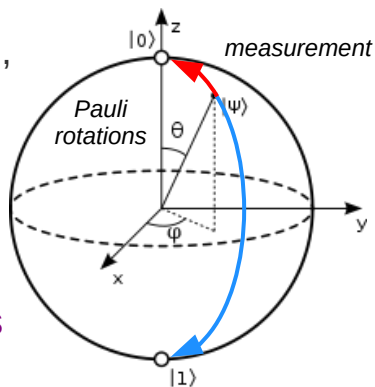
Parameterized Quantum Circuits (PQC) & Variational Quantum Algorithms (VQA)

In PennyLane a **Circuit** is a function.
To become executable on a specific device,
it needs to become a **QNode** (quantum node).



We create a **PQC** (variational model) = a circuit template with parameterised gates, e.g. **Rx(a)**, **Ry(a)** or **Rz(a)** or **P(a)**, each allowing rotation of a qubit state in x, y or z axis (as per **Bloch sphere**).

Typically (but now always), circuits consist of three functional blocks, i.e. **feature map**, **ansatz** and **measurements** (not necessarily in this order).



Feature map encodes (embeds) classical input data into its input parameters, setting the model's initial quantum state.

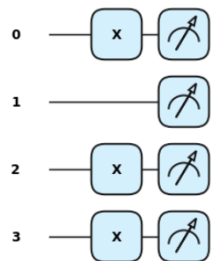
Ansatz evolves the quantum state, it consists of trainable quantum gates, weight params trained by a classic optimiser

Measurements determine the circuit final state and convert it into the model's output in the form of classical data.

Data Encoding

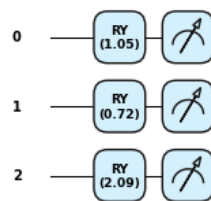
Maria Schuld and Francesco Petruccione
Machine Learning with Quantum Computers.
2nd ed. Springer, 2021.

Basis encoding



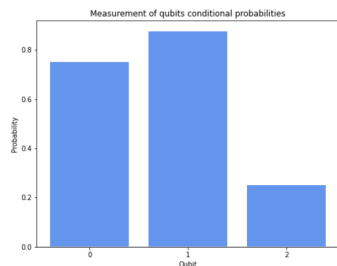
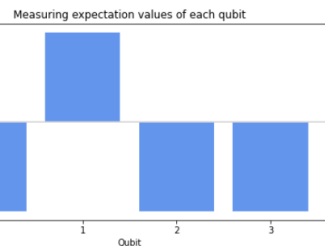
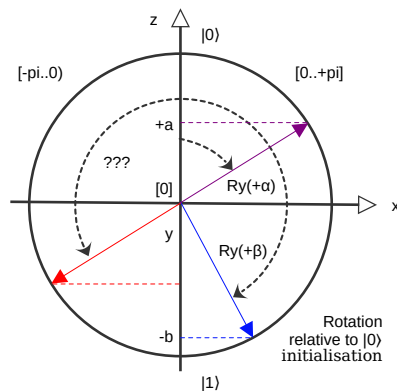
The simplest data encoding and very popular, however, little data can be encoded at a time, you can encode only binary values per each qubit.

Angle encoding



One of the most flexible and very effective, you can encode floating point numbers.

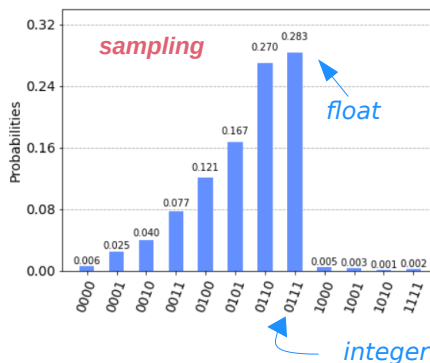
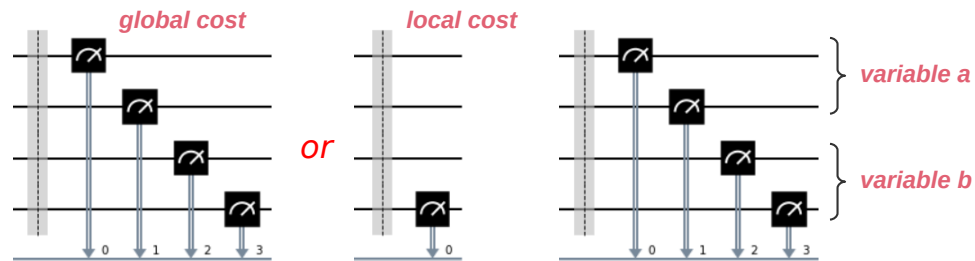
What you encode depends on your intention!



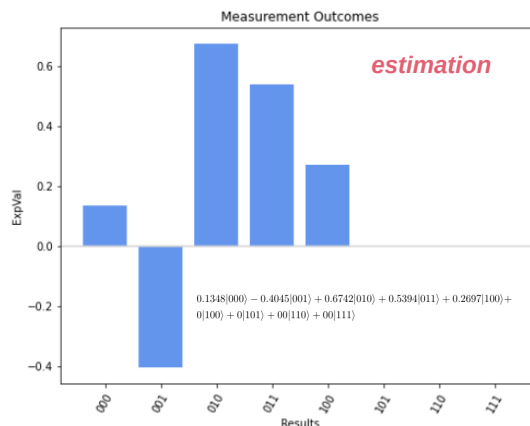
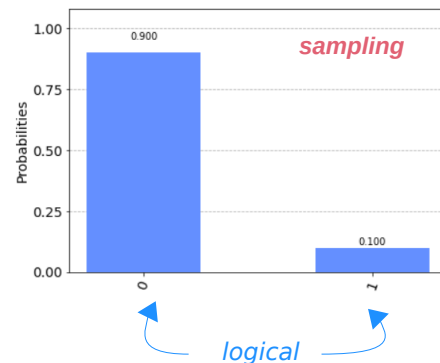
Encoded values:
 $\arccos(0.5)$
 $\arccos(0.75)$
 $\pi - \arccos(0.5)$

There are many encoding methods, e.g. basis, angle, amplitude, QRAM, ...

Encoding can be repeated across the circuit, which is called data reuploading



or



Measurements must be repeated, collected and then can be interpreted in many different ways

It is also possible to measure mid-circuit, however, beware as the circuit is no longer unitary and not reversible!

State Measurement

Why PennyLane?

Everything is a function!



PennyLane (PL) ...

- Supports *differentiable programming paradigm*
- Integrates seamlessly with *Python*
- Has a range of operations for *state preparation*, *gates*, *measurements* and *models*
- Supports creation of flexible *quantum algorithms*
- Executes on *simulators* and *quantum hardware*
- Supports *error mitigation*
- Extends its built-in *system of gradients* with those from JAX, PyTorch, Keras, TensorFlow, or NumPy
- Allows training with *hardware-compatible gradients* and *higher-order derivatives*
- Supports *hybrid quantum-classical models*
- Provides just in time *compilation* and *optimisation*
- Can be extended with models and optimisers from other SDKs, e.g. *PyTorch* and *TensorFlow*

PennyLane Demo:

- Create a simple PL model to fit a simple function
- Learn to initialise model weights
- Explore the impact of ansatz structure on performance
- Create minimalistic quantum models
- Learn the interaction of data encoding and ansatz
- Investigate different types of entangling
- Apply the best solution to more complex data
- Learn about stamina and wisdom in QML development

Key takeaways:

- Plan model development, tests and experiments
- Bad data encoding spoils the bunch!
- Strong entanglement improves the data fit
- More width and depth = the curse of dimensionality
- Carefully consider your quantum model initialisation
- Surprise - a single qubit model still works! (and well)
- More training does not solve the problems
- Data reuploading makes a huge difference!

Training a PennyLane model

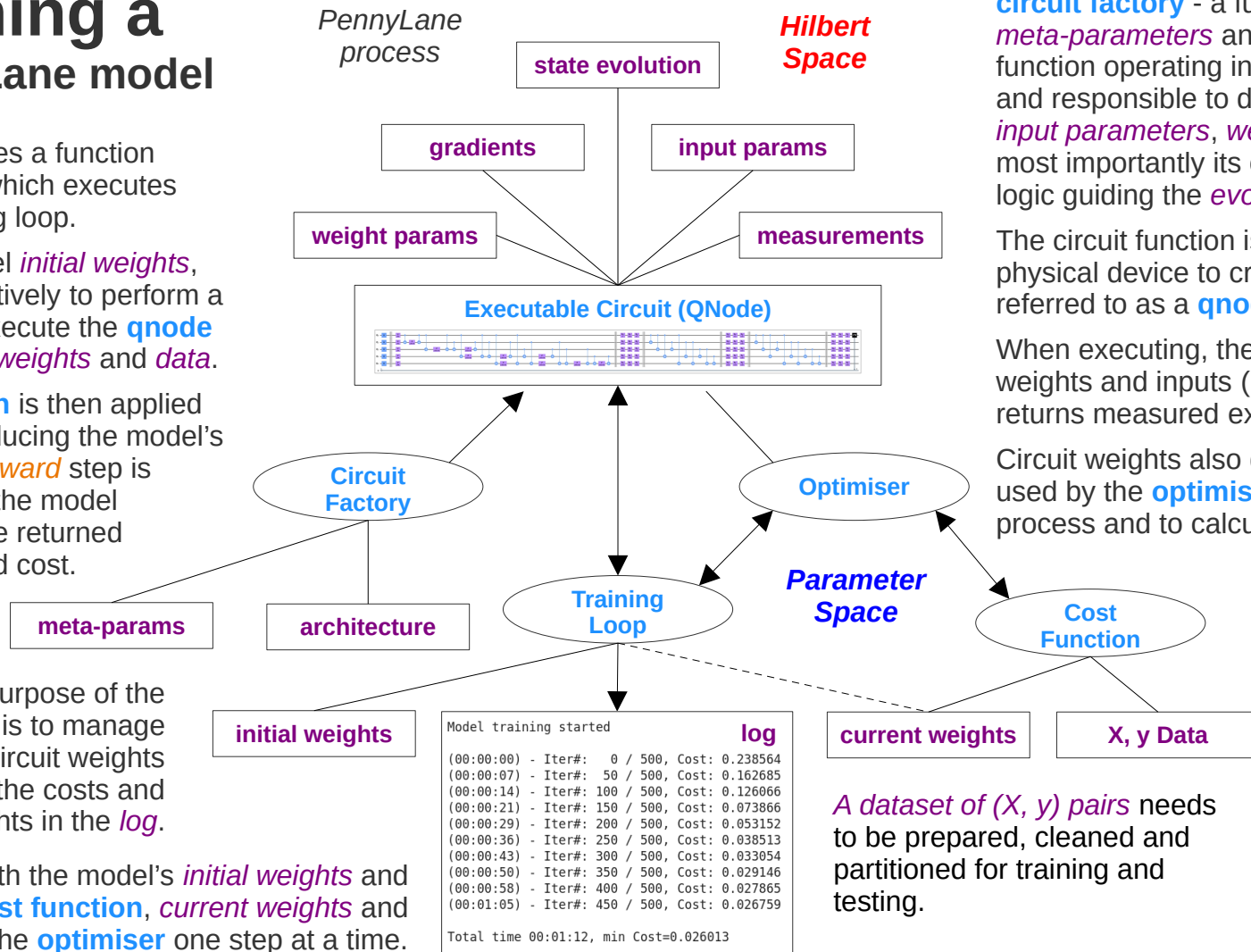
Optimiser provides a function `step_and_cost` which executes a step in a training loop.

It starts with model *initial weights*, and is called iteratively to perform a *forward* step to execute the **qnode** using the *current weights* and *data*.

The **cost function** is then applied to the results producing the model's cost. Then a *backward* step is taken to improve the model weights, which are returned with the calculated cost.

The purpose of the **training loop** is to manage and improve the circuit weights and collect the costs and models weights in the *log*.

The loop starts with the model's *initial weights* and then passes the **cost function**, *current weights* and *input data* to the **optimiser** one step at a time.



In PennyLane the PQC is typically written as a **circuit factory** - a function defining the circuit *meta-parameters* and generating a **closure**, a function operating in the factory's environment, and responsible for defining the circuit *input parameters*, *weight parameters*, and most importantly its overall *architecture* and logic guiding the *evolution of its state*.

The circuit function is then deployed on a physical device to create an executable circuit, referred to as a **qnode** (quantum node).

When executing, the qnode takes values of weights and inputs (possibly in batches) and returns measured expectation values.

Circuit weights also define *gradients* which are used by the **optimiser** during the optimisation process and to calculate expectation values.

The qnode can be deployed on a variety of devices (e.g. a simulator "default.qubit", GPU "lightning.gpu" or real QPU, e.g. "ionq.qpu"), and optionally interfaced with various gradient packages (e.g. "torch") and differentiation methods (e.g. "adjoint").

A dataset of (X, y) pairs needs to be prepared, cleaned and partitioned for training and testing.

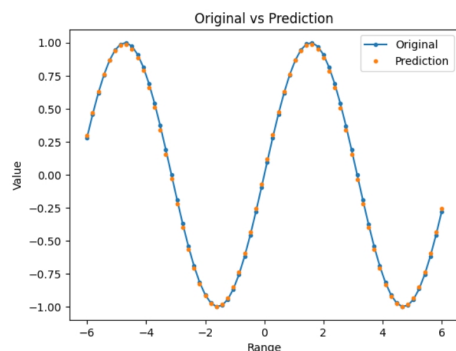
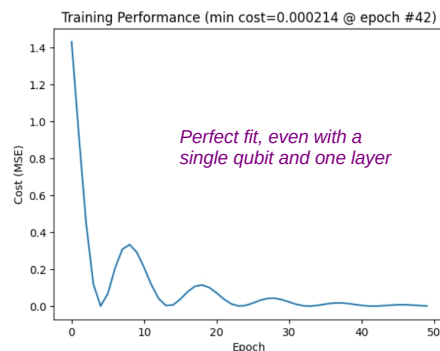
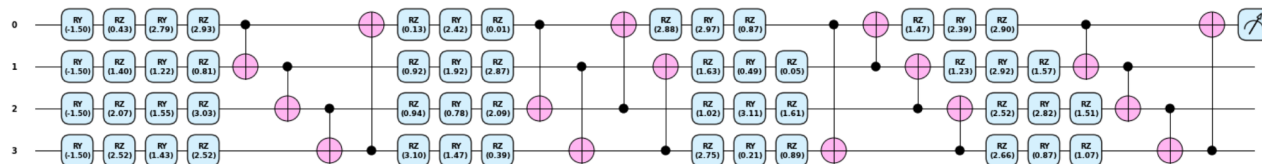
Sample Model Training:

Sweat and tears (synthetic data)

The minimal, near-optimal circuit



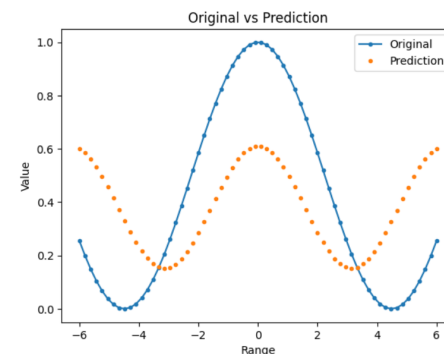
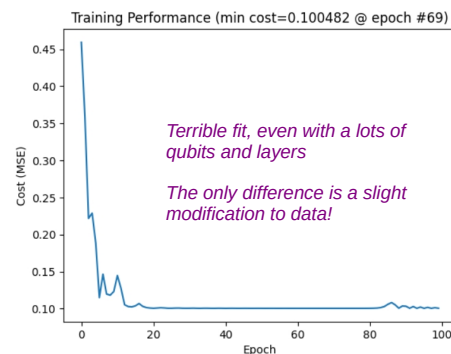
The best circuit for the first dataset, but not working for the second dataset



Training completed: epochs=50, min cost=0.000214 @ 42, time=002 secs

*We have optimised the model for a simple dataset!
It worked perfectly well for this data but failed to
work on a similar data set!*

Why did it work so well?



Training completed: epochs=100, min cost=0.100482 @ 69, time=028 secs

*How can we improve this model for the new dataset?
We experimented with weight initialization, the number of
epochs, qubits, layers, entanglement types? All failed!*

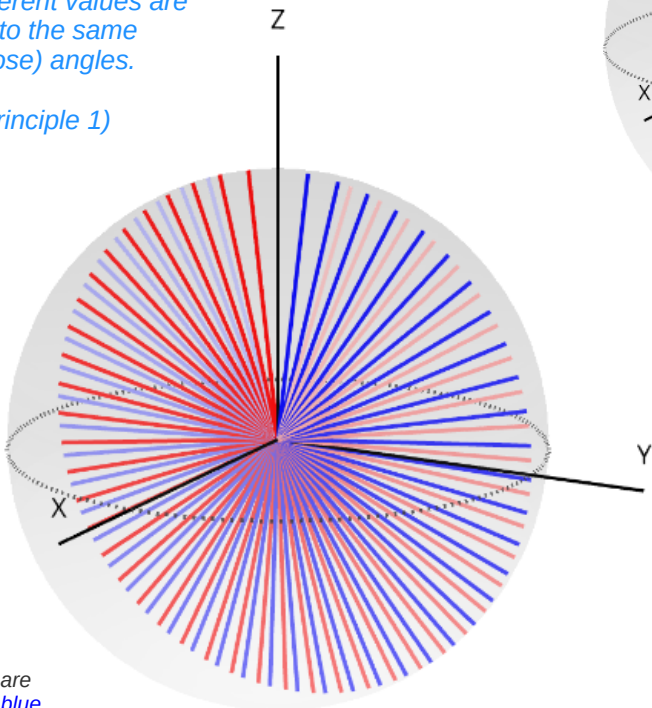
Why did it fail so badly?

Angle encoding

The Good, the Bad and the Ugly

This example shows encoding values wrapping around the Bloch sphere (e.g. -6 to 6), so that different values are mapped into the same (or very close) angles.

(violates principle 1)

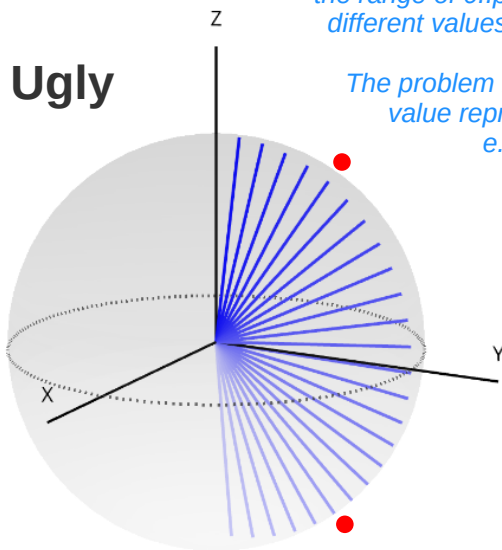


Positive values are represented by blue vectors and negative values by red vectors.

This example shows encoding values wrapping around the range of $0..pi$ of the Bloch sphere, ensuring that different values are represented by unique angles.

The problem may arise if we may have the same value represented by two distinct amplitudes, e.g. values of $\sin(x)$ represented by x

(violates principle 2)



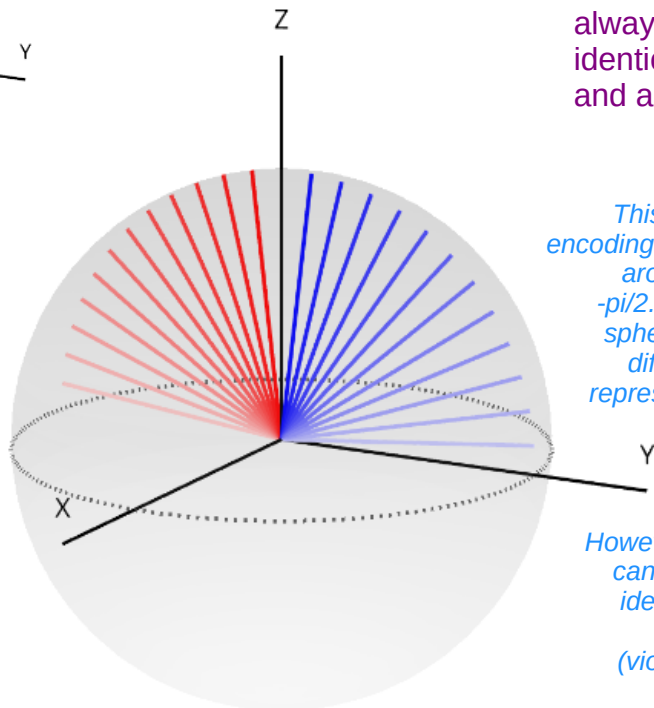
Special care must be taken in those cases where the values form some kind of symmetry or repetition. e.g. in a case values x and $2\pi+x$ effectively represent the same angular position and their encoding should also be identical.

The red dot represents the same value which on two occasions maps onto two different angles and thus two amplitudes.

Two principles of quantum data encoding:

- 1) distinct data values should map onto distinct angles and amplitudes
- 2) the same data values should always map into identical angles and amplitudes

This example shows encoding values wrapping around the range of $-\pi/2..pi/2$ of the Bloch sphere, ensuring that different values are represented by unique angles.

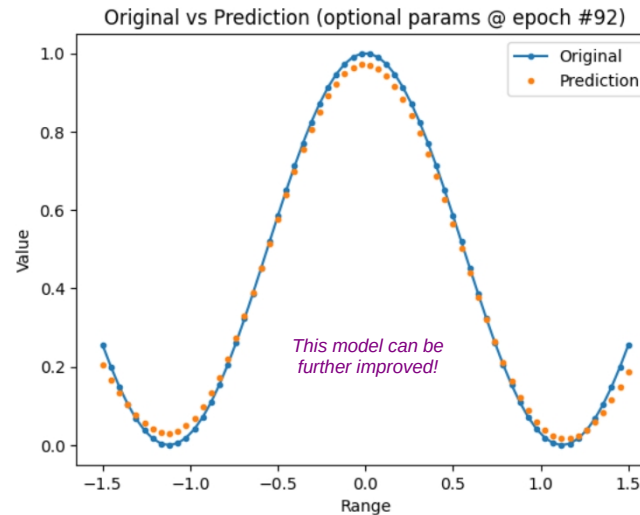
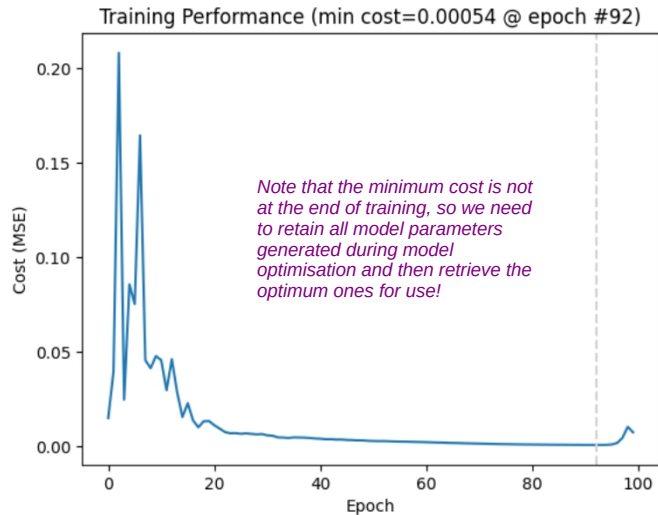
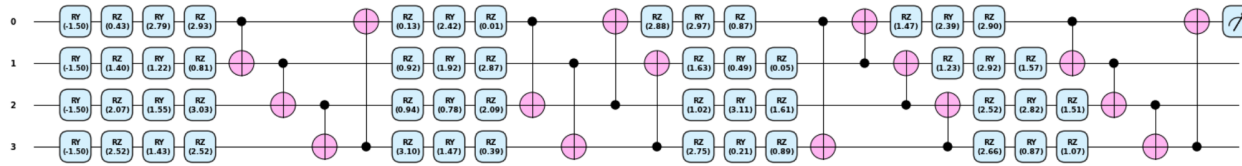


However, some angles can be measured as identical amplitudes

(violates principle 1)

Improved model

The same circuit as the best circuit used with the first dataset



Training completed: epochs=100, min cost=0.00054 @ 92, time=029 secs

Corrected / improved the model:

- Corrected data encoding to map into the valid rotation range
- Introduced a simple data reuploading (across wires)
- Logging model parameters during optimisation, so that we could use the best rather than last model

Improved code / presentation quality:

- Implemented the cost function and a circuit as closures (no globals)
- Created a model shape function to match model factory function (no more code replication)
- Used PennyLane standard functions for creating encoding and entangling blocks
- Increased flexibility by separating circuit and qnode creation
- Parameterised all aspects of models creation and training (no hard-coded constants)
- Changed circuit drawing to graphical representation
- Improved presentation of the chart plotting training costs

Recommended reading on QML with Qiskit / PennyLane

Quantum Science and Technology

Maria Schuld
Francesco Petruccione

Machine Learning with Quantum Computers

Second Edition

Springer

EXPERT INSIGHT

Dancing with Qubits

From qubits to algorithms, embark on the quantum computing journey shaping our future

Second Edition



Robert S. Sutor

A Practical Guide to Quantum Machine Learning and Quantum Optimization

Hands-on Approach to Modern Quantum Algorithms

ELÍAS F. COMBARRO
SAMUEL GONZÁLEZ-CASTILLO

Foreword by Alberto Di Meglio,
Head of Innovation - Coordinator CERN Quantum Technology Initiative

Qiskit Machine Learning: an open-source library for quantum machine learning tasks at scale on quantum hardware and classical simulators

M. Esmat Sahin¹, Edoardo Arnesen², Oscar Wallin³, Stephen P. Wood⁴, Anton Delamare⁵,
Devlin A. Miller⁶, Tamas Baumgratz⁷, Anand Marathe⁸, Stefan Brannas⁹, and Co-ordinators
¹The Qiskit Center, EPFL, 8003 St-James, Dübendorf, Switzerland, ²IBM Research, Yorktown Heights, NY 10598, USA
³IBM Quantum, IBM, Watson Research Center, Yorktown Heights, NY 10598, USA
⁴IBM Quantum, IBM Research Europe, Zürich, Switzerland
⁵IBM Quantum, IBM Research Europe, Zürich, Switzerland
⁶IBM Quantum, IBM Research Europe, Zürich, Switzerland
⁷IBM Quantum, IBM Research Europe, Zürich, Switzerland
⁸IBM Quantum, IBM Research Europe, Zürich, Switzerland
⁹IBM Quantum, IBM Research Europe, Zürich, Switzerland

We present Qiskit Machine Learning (QML), a high-level Python library that enables elements of quantum computing, with traditional machine learning (ML), to be integrated into a single framework. The ML framework is designed to be a modular and extensible, making it easy to integrate with existing ML frameworks. The QML framework is designed to be a modular and extensible, making it easy to integrate with existing ML frameworks. The QML framework is designed to be a modular and extensible, making it easy to integrate with existing ML frameworks.

1. INTRODUCTION
The convergence of quantum computing and machine learning presents a perspective shift in both research and industry. Quantum machine learning (QML) leverages the principles of quantum mechanics to potentially enhance or accelerate classical machine learning algorithms, opening new frontiers in fields ranging from materials science to finance. As the field of QML matures, there is a growing need for accessible and powerful software tools that bridge the gap between theoretical QML algorithms and their practical implementation on emerging quantum hardware and simulators.

2. DESIGN STRUCTURE
This section outlines the role of Qiskit ML within the broader quantum computing software stack, as illustrated in Fig. 1. Qiskit ML sits at the application level, providing a suite of tools and algorithms that leverage the power of quantum computing for machine learning tasks.

Qiskit Machine Learning (QML) is an open-source module within the Qiskit framework [1], addressing this need by providing a comprehensive and user-friendly platform for exploring the existing landscape of QML. Built on top of Qiskit's core stack, QML provides a high-level interface for quantum circuit design, simulation, and execution, as well as a bridge between high-level machine learning concepts and the underlying quantum hardware or simulators. The design of Qiskit ML prioritizes extensibility, modularity, and an intuitive user interface, built on top of a robust and well-maintained quantum computing stack. The design of Qiskit ML prioritizes extensibility, modularity, and an intuitive user interface, built on top of a robust and well-maintained quantum computing stack.

Qiskit ML was started in 2017 as an open-source library for quantum computing by IBM. Six years after its initial release, the Qiskit ecosystem is thriving. The package has been installed over 6 million times, at a current rate of 200,000 per month. 500+ individuals have contributed to the codebase, and the user community is growing rapidly. The package has been installed over 6 million times, at a current rate of 200,000 per month. 500+ individuals have contributed to the codebase, and the user community is growing rapidly.

Modern applications of machine learning in quantum sciences

Anna David^{1,2,3,4}, Julian Arnold⁵, Barja Requena⁶, Alexander Gresh^{6,7}, Marcin Płodzień⁸, Kaelan Donatelli⁹, Kim A. Nicol¹⁰, Paolo Storzati¹¹, Rouven Koch¹⁰, Miriam Böttner¹¹, Robert Okula^{12,13}, Gorika Muljoo-Gil¹⁴, Rodrigo A. Vargas-Hernández^{15,17}, Alba Cervera-Lierta¹⁶, Juan Carrasquilla¹⁷, Vedran Dunjko¹⁸, Maynard Gehring¹⁹, Patrick Humbel^{20,22}, Evert van Nieuwenburg^{10,23}, Filippo Vicentini^{24,25}, Lei Wang^{25,26}, Sebastian J. Wernitz²⁷, Giuseppe Carleo²⁸, Erika Greplova²⁹, Roman Krems³⁰, Florian Marquardt^{31,32}, Michael Tomate³³, Marcel Lewenstein^{34,35} and Alexandre Dauphin^{36,37}

- ¹ Faculty of Physics, University of Warsaw, Poland
- ² ICFO - Institut de Ciències Fotòniques, The Barcelona Institute of Science and Technology, 08860 Castelldefels (Barcelona), Spain
- ³ Center for Computational Quantum Physics, Flatiron Institute, New York, USA
- ⁴ Department of Physics, University of Basel, Switzerland
- ⁵ Institute of Theoretical Physics, Heinrich Heine University of Düsseldorf, Germany
- ⁶ Institute for Quantum Optics and Quantum Information, Hamburg University of Technology, Germany
- ⁷ Université de Paris, CNRS, Laboratoire Matériaux et Photophysique Quantiques, France
- ⁸ Machine Learning Group, Technische Universität Berlin, Germany
- ⁹ IFQD, Berlin Institute for the Foundations of Learning and Data, 10587 Berlin, Germany
- ¹⁰ Department of Applied Physics, Aalto University, Espoo, Finland
- ¹¹ Institute of Physics, Heinrich Heine University of Düsseldorf, Germany
- ¹² International Center for Theory of Quantum Technologies, University of Gdańsk, Poland
- ¹³ Department of Applied Physics and Photonics, Faculty of Electronics, Faculty of Electronics, Telecommunications and Informatics, Gdańsk University of Technology, Poland
- ¹⁴ Institute for Theoretical Physics, University of Innsbruck, Austria
- ¹⁵ Department of Chemistry, University of Toronto, Canada
- ¹⁶ Vector Institute for Artificial Intelligence, MARS Centre, Toronto, Canada
- ¹⁷ Department of Chemistry and Chemical Biology, McMaster University, Hamilton, Canada
- ¹⁸ Barcelona Supercomputing Center, Spain
- ¹⁹ IACS, Leiden University, The Netherlands
- ²⁰ CNRS, Ecole Polytechnique, France
- ²¹ Institute of Physics, Ecole Polytechnique Fédérale de Lausanne (EPFL), Switzerland
- ²² Max Planck Institute, Palo Alto, California, United States of America
- ²³ Niels Bohr Institute, Copenhagen, Denmark
- ²⁴ CPT, CNRS, Ecole Polytechnique, France
- ²⁵ Beijing National Lab for Condensed Matter Physics and Institute of Physics, Chinese Academy of Sciences, Beijing, China
- ²⁶ Stockholm Lake Materials Laboratory, Sweden
- ²⁷ Princeton Institute for Theoretical Physics, Princeton, USA
- ²⁸ Kavli Institute of Nanoscience, Delft University of Technology, 2600 CA Delft, The Netherlands
- ²⁹ Department of Chemistry, University of British Columbia, Vancouver, Canada
- ³⁰ Max Planck Institute for the Science of Light, Erlangen, Germany
- ³¹ Department of Physics, Friedrich-Alexander-Universität Erlangen-Nürnberg, Germany
- ³² ICFO, Pg. Jàume Guardià 23, 08010 Barcelona, Spain
- ³³ IRSO, SAS, 7 rue Leonardo da Vinci, 93100 Marly, France

¹ These authors contributed equally.
² * a.david@icfo.eu, alexandre.dauphin@epfl.ch

September 21, 2025

PennyLane: Automatic differentiation of hybrid quantum-classical computations

Ville Bergholm¹, Josh Isaac², Maria Schuld³, Christian Gogolin⁴, M. Sohaib Alam⁵, Shantanu Ahmed⁶, Juan Miguel Arraiza⁷, Carson Blank⁸, Alain Delgado⁹, Soran Jahangiri¹⁰, Keni McKernan¹¹, Johannes Jakob Meyer¹², Zeyue Xu¹³, Anirudh Saha¹⁴, and Nathan Kilian¹⁵

¹QinetiQ, 777 Bay Street, Toronto, Canada
²QinetiQ Computing, 2019 Seventh Street, Berkeley, CA 94710
³Qiskit Center for Quantum Technology, Department of Microtechnology and Nanoscience, Chalmers University of Technology, 412 96 Göteborg, Sweden
⁴QinetiQ, 777 Bay Street, Toronto, Canada
⁵QinetiQ, 777 Bay Street, Toronto, Canada
⁶QinetiQ, 777 Bay Street, Toronto, Canada
⁷QinetiQ, 777 Bay Street, Toronto, Canada
⁸QinetiQ, 777 Bay Street, Toronto, Canada
⁹QinetiQ, 777 Bay Street, Toronto, Canada
¹⁰QinetiQ, 777 Bay Street, Toronto, Canada
¹¹QinetiQ, 777 Bay Street, Toronto, Canada
¹²QinetiQ, 777 Bay Street, Toronto, Canada
¹³QinetiQ, 777 Bay Street, Toronto, Canada
¹⁴QinetiQ, 777 Bay Street, Toronto, Canada
¹⁵QinetiQ, 777 Bay Street, Toronto, Canada

PennyLane is a Python 3 software framework for optimization and machine learning on quantum and hybrid quantum-classical computations. The library provides a unified architecture for near-term quantum computing devices, supporting both gate and continuous-variable paradigms. Its primary aim is to enable the development of quantum machine learning algorithms in a way that is compatible with classical techniques such as backpropagation. PennyLane thus extends the automatic differentiation algorithms common in optimization and machine learning to include quantum and hybrid computations. A hybrid quantum machine learning framework compatible with gate-based quantum circuit simulators or hardware, the library provides a high-level interface for quantum circuit construction, optimization, and execution. PennyLane thus extends the automatic differentiation algorithms common in optimization and machine learning to include quantum and hybrid computations.

Introduction

Recent progress in the development and commercialization of quantum technologies has had a profound impact on the landscape of quantum algorithms. Near-term quantum devices enable routine use of shallow-depth and robust against errors. The design paradigm of hybrid algorithms which integrate classical and quantum processing has therefore become increasingly important. Recently the most well-known class of hybrid algorithms, quantum circuits that are optimized by a classical computer with regards to a given objective.

Hybrid optimization with variational circuits opens up a number of new research avenues for near-term quantum computing.

Quantum computing with Qiskit

Ali Javadi-Abadi¹, Matthew Treinish², Kevin Krawinkel³, Christopher J. Wood⁴, Jake Lishman⁵, Julien Gaerin⁶, Simon Martiel⁷, Paul D. Nation⁸, Leo S. Bishop⁹, Andrew W. Cross¹⁰, Blake R. Johnson¹¹, and Jay M. Gambetta¹²
¹IBM Quantum, IBM Research Europe, Zürich, Switzerland
²IBM Quantum, IBM Research Europe, Zürich, Switzerland
³IBM Quantum, IBM Research Europe, Zürich, Switzerland
⁴IBM Quantum, IBM Research Europe, Zürich, Switzerland
⁵IBM Quantum, IBM Research Europe, Zürich, Switzerland
⁶IBM Quantum, IBM Research Europe, Zürich, Switzerland
⁷IBM Quantum, IBM Research Europe, Zürich, Switzerland
⁸IBM Quantum, IBM Research Europe, Zürich, Switzerland
⁹IBM Quantum, IBM Research Europe, Zürich, Switzerland
¹⁰IBM Quantum, IBM Research Europe, Zürich, Switzerland
¹¹IBM Quantum, IBM Research Europe, Zürich, Switzerland
¹²IBM Quantum, IBM Research Europe, Zürich, Switzerland

We describe Qiskit, a software development kit for quantum information sciences. We discuss the key design decisions that have shaped its development, and examine the software architecture and core components. We demonstrate an end-to-end workflow for solving a problem in condensed matter physics on a quantum computer that serves to highlight some of Qiskit's capabilities, for example the representation and optimization of circuits as various abstraction levels, the scalability and reusability of new gates, and the use of quantum-classical circuits as dynamic circuits. Lastly, we discuss some of the concepts and tools and plugins that extend Qiskit for various tasks, and the future outlook.

1. INTRODUCTION

Quantum computing is progressing at a rapid pace, and robust software tools such as Qiskit are becoming increasingly important as a means of facilitating research, education, and to run computationally interesting problems on quantum computers. For example, Qiskit was used in a recent paper that showed evidence of utility for quantum computing by noise error mitigation [24, 60]. Circuits at this level can be quite abstract, for example only specifying a set of Pauli rotations, some unitaries, or other high-level mathematical operators. Importantly, these abstract circuits are reproducible in Qiskit, which contains methods to generate concrete circuits from them. Such concrete circuits are formed using a standard library of gates, representable using intermediate quantum languages such as OpenQASM [24].

The transformer rewrites circuits in multiple rounds of passes, in order to optimize and translate it to the target instruction set architecture (ISA). The word "transformer" is used within Qiskit to emphasize its nature as a circuit-to-circuit rewriting tool, distinct from a full compilation down to controller hardware which is necessary for executing circuits. But the transformer can also be thought of as an optimizing compiler for quantum programs.

The ISA is the key abstraction layer separating the hardware from the software, and depends heavily on the quantum computer architecture backend. For example for a physical quantum computer based on superconducting qubits, this can include CNOT, X and Z gates. For a logical quantum computer, it can include joint Pauli measurements, magic state distillation, or other operations specific to the error correcting code [25]. Note that the ISA is often more than just a universal set of quantum gates, and can include measure, reset or de-purify operations, or classical control-flow such as if/else/while loops and for/while loops, or limited conditional instructions on classical data. For example in Appendix B for the current ISA pertaining to IBM's quantum computers.

arXiv:2204.04198v4 [quant-ph] 7 Jun 2025

arXiv:2002.08080v3 [quant-ph] 14 Feb 2020

Summary and Q&A

- QML is an intersection of QC x ML x Maths
- The most common approach to PQC training are VQAs
- There exist a variety of different methods of quantum encoding
- There exist a variety of different ways of interpreting state measurements
- When training a quantum model log costs and model parameters - you may need both later on (e.g. scoring training / testing models)
- Simple data may lead to deceitful feel of success!
- Bad data encoding is the harbinger of a good model's death!
- What are the principles of quality PennyLane code?
 - Encapsulate models and cost functions in closures
 - Keep model shape function near the model definition functions
 - Data reuploading often greatly improves model accuracy
 - Use PennyLane standard functions (great way to cut your effort)
 - Parameterised all aspects of models creation and training



Jacob Cybulski,
Founder, Enquanted, Australia
Hon. Assoc. Prof. Deakin, SIT

Jacob's recent projects:

Quantum Information Field Theory (QIFT)
combining Quantum Field Theory with Information Processing

Geometry of the Hilbert space
in hybrid quantum-classical models

Quantum time series analysis
processing of quantum time series and signals, e.g. data encoding, measurement, and model training (such as QNNs, QAEs, QRC, etc.)

Quantum model expressivity and trainability
balancing the model's ability to effectively represent data in the Hilbert space, as well as, its capacity to learn and generalize
Impact of barren plateaus countermeasures on QNN capacity to learn
incl. methods of measuring its capacity

Quantum graph theory
exploration of quantum representation and processing of graphs
e.g. identifying clusters and communities

Foundations of quantum machine learning
Business value of quantum computing and QML

Available resources:

<https://jacobcybulski.com/>
<https://github.com/ironfrown/>



*This presentation has been released under the
Creative Commons CC BY-NC-ND license, i.e.*

BY: credit must be given to the creator.
NC: Only noncommercial uses of the work are permitted.
ND: No derivatives or adaptations of the work are permitted.